



第四章 承诺和零知识证明

目 录

「01」

知识点一：哈希承诺

「02」

知识点二：加法同态承诺

「03」

知识点三：零知识证明

「04」

知识点四：零知识证明应用示例

1. 承诺

- 在日常生活中，承诺无处不在。
- 例如，预约打车成功后，司机和乘客之间就互相做了一个承诺。到了预约时间，乘客等车，司机接客，这就是在兑现承诺。
- 一般而言，密码学中的承诺涉及**承诺方**和**验证方**，它**允许承诺方向验证方对一个秘密值做出承诺，承诺方后续会向验证方披露此秘密值。**

1. 承诺

➤ 承诺协议包含两个阶段：

承诺生成 (Commit) 阶段： 承诺方选择一个**敏感数据** v ，计算出对应的**承诺** c ，然后将承诺 c 发送给验证方。通过承诺 c ，验证方确定承诺方对于还未解密的敏感数据 v 只能有唯一的解读方式，无法违约。

承诺披露 (Reveal) 阶段： 也称为承诺打开-验证 (Open-Verify) 阶段。承诺方**公布敏感数据** v 的明文和其他的相关参数，验证方重复承诺生成的计算过程，比较新生成的承诺与之前接收到的承诺 c 是否一致，一致则表示验证成功，否则失败。

1. 承诺

➤ **承诺有两个性质：**

隐藏性 (Hiding)： 在承诺方未向接收方披露承诺值之前，**验证方无法得到与承诺值相关的任何信息**，这一性质称为隐藏性。

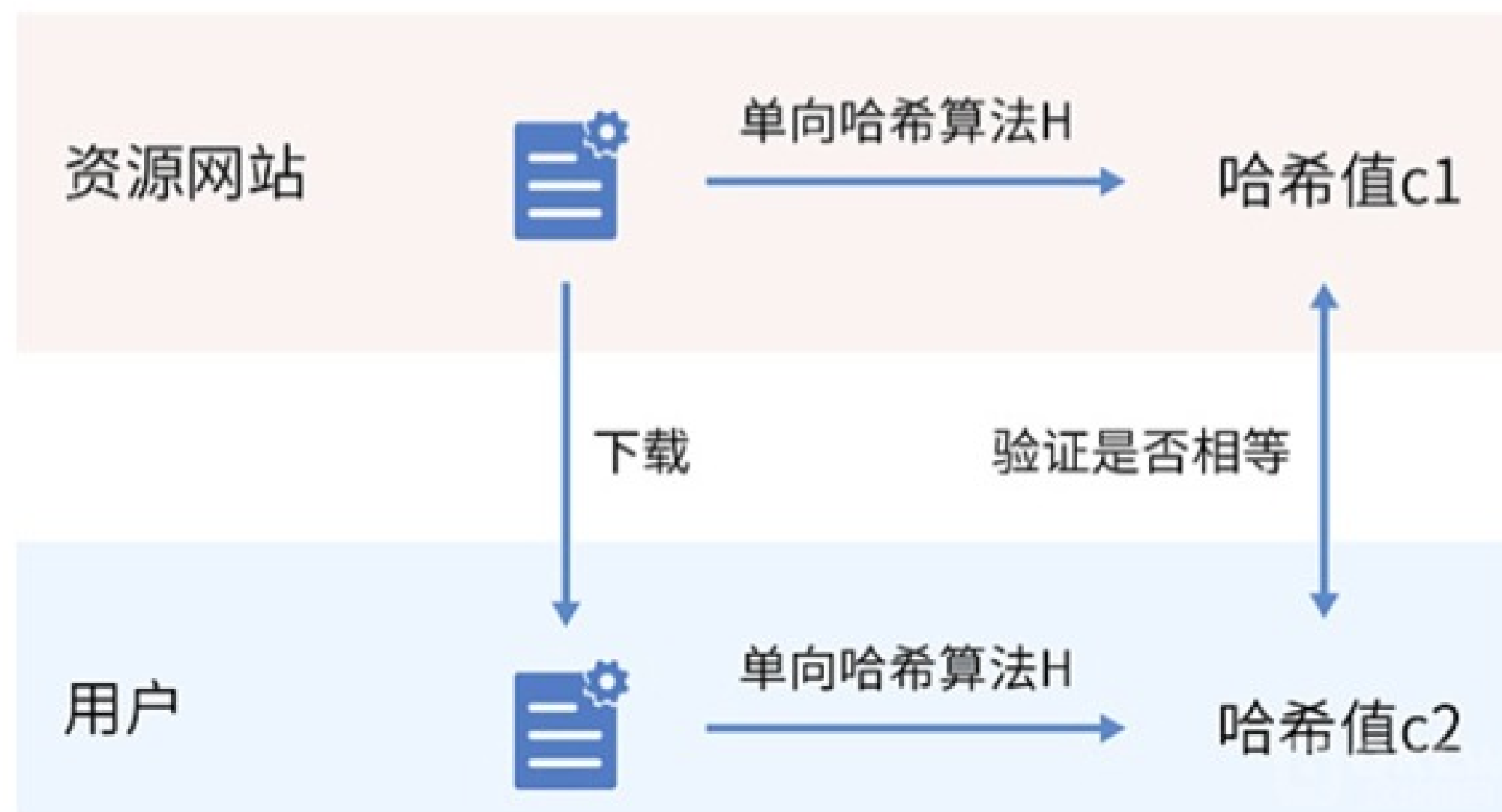
绑定性 (Binding)： 在对秘密值做出承诺之后，**承诺方无法对秘密值进行任何修改**，这一性质称为绑定性。

1. 承诺

- 哈希承诺就是一个简单的承诺协议。
- 承诺生成阶段：将通过输入的消息 v 的哈希值 $H(v)$ 作为承诺并公开；
- 承诺披露阶段：将对获取的消息 v' 通过计算它的哈希值 $H(v')$ 并与之前公开的承诺 $H(v)$ 进行比对完成验证。
- 基于单向哈希的单向性，难以通过哈希值 $H(v)$ 反推出敏感数据 v ，以此提供了一定的隐匿性；
- 基于单向哈希的抗碰撞性，难以找到不同的敏感数据 v' 产生相同的哈希值 $H(v)$ ，以此提供了一定的绑定性。

1. 承诺

- **典型应用。** 某些网站在提供下载文件时，会提供对应文件的单向哈希值。这里，单向哈希值便是一种对文件数据的承诺。基于下载的哈希承诺，用户可以对下载文件数据进行校验，检测接收到的文件数据是否有丢失或变化，如果校验通过，相当于网站兑现了关于文件数据完整性的承诺。



1. 承诺

提升安全性的哈希承诺。

- 对隐私数据机密性要求高的应用，需要注意哈希承诺提供的隐匿性比较有限，不具备随机性。对于同一个敏感数据 v ， $H(v)$ 值总是固定的，因此可以通过暴力穷举，列举所有可能的 v 值，来反推出 $H(v)$ 中实际承诺的 v 。
- 利用随机预言机模型可以构造更加安全高效承诺协议。如果想对 x 做出承诺，只需要简单地选择一个随机值 $r \in_R \{0,1\}^k$ 并公布 $y = H(x \parallel r)$ 。后续只需要简单地披露 x 和 r 即可。

使用哈希承诺的时候，不同的场景尽可能引入不同的随机数，公布消息和随机数的哈希值作为承诺；在承诺披露的阶段，公布消息和随机数即可。这种做法是否正确？

- ☒ A 是
- ☐ B 否

目 录

「01」

知识点一：哈希承诺

「02」

知识点二：加法同态承诺

「03」

知识点三：零知识证明

「04」

知识点四：零知识证明应用示例

1. 加法同态承诺

- **哈希承诺不具备同态特性**，多个相关的承诺值之间无法进行密文运算和交叉验证，复杂的验证都需要承诺披露后才能进行，对于构造复杂密码学协议和多方安全计算方案的作用比较有限。
- **同态承诺** (Homomorphic Commitment) 与同态加密概念相似，可以在公开的承诺上直接执行一定的计算，使得其与明文上运算后的承诺一致。

1. 加法同态承诺

基本定义：

加法同态承诺是具有加法同态性质的同态承诺，即给定承诺 $\text{Com}(x; r_x)$ 和 $\text{Com}(y; r_y)$ ，存在运算 $\oplus$ ，满足

$$\text{Com}(x; r_x) \oplus \text{Com}(y; r_y) = \text{Com}(x+y; r_x+r_y)。$$

可以这么解释，对于两个敏感数据 x 和 y 的承诺执行运算 $\oplus$ ，与两个敏感消息先执行 $+$ 之后再生成的承诺是相同的，都可以在承诺披露阶段完成验证。

2. Pedersen承诺

Pederson同态承诺（简称Pedersen承诺）是由 Torben Pryds Pedersen于1992年提出的一种承诺。Pedersen承诺主要搭配椭圆曲线密码学使用，具有基于离散对数困难问题的强绑定性以及同态加法特性。

(1) 椭圆曲线

椭圆曲线上的点可以进行加、减或乘（乘以整数，也称标量）运算。

椭圆曲线上的点满足以下特性：

- 给定一个整数 k ，其可以与曲线上的点 G 进行标量乘法运算，即 kG ，所得结果也是曲线上的一个点。通过 kG 反推 k 是不可能的（椭圆曲线离散对数求解难题）。
- 给定另一个整数 j ， $(k+j)G$ 等于 $kG + jG$ ，即椭圆曲线上的加法和标量乘法运算保持加法和乘法的交换率和结合律。

2. Pedersen承诺

(2) 具体构造

初始化阶段：初始化椭圆曲线，并选择曲线上的两个点 G 和 H 。

承诺阶段：承诺方选择随机数 r 作为盲因子，对隐私信息 v 计算承诺值

$$C = r G + v H,$$

并发送给接收方。

披露阶段：承诺方发送 (v, r) 给接收者，接收者验证

$$C \text{ 是否等于 } r G + v H,$$

如果相等则接受，否则拒绝承诺。

2. Pedersen承诺

(3) 加法同态性

Pedersen承诺具备加法同态特性，这是椭圆曲线点运算的性质决定的。

假设有两个要承诺的信息 v_1 、 v_2 ，随机数 r_1 、 r_2 ，生成对应的两个承诺为：

$$C(v_1) = r_1G + v_1H, C(v_2) = r_2G + v_2H$$

显然，满足加法同态特性：

$$C(v_1) + C(v_2) = (r_1G + v_1H) + (r_2G + v_2H) = (r_1 + r_2)G + (v_1 + v_2)H = C(v_1 + v_2)。$$

3. 应用示例

Pedersen承诺有很多用途，允许我们承诺一个信息，但在未来的某个时候才真正透露出来。我们还可以使用Pedersen承诺来实现一些不解密隐私信息时候的验证。

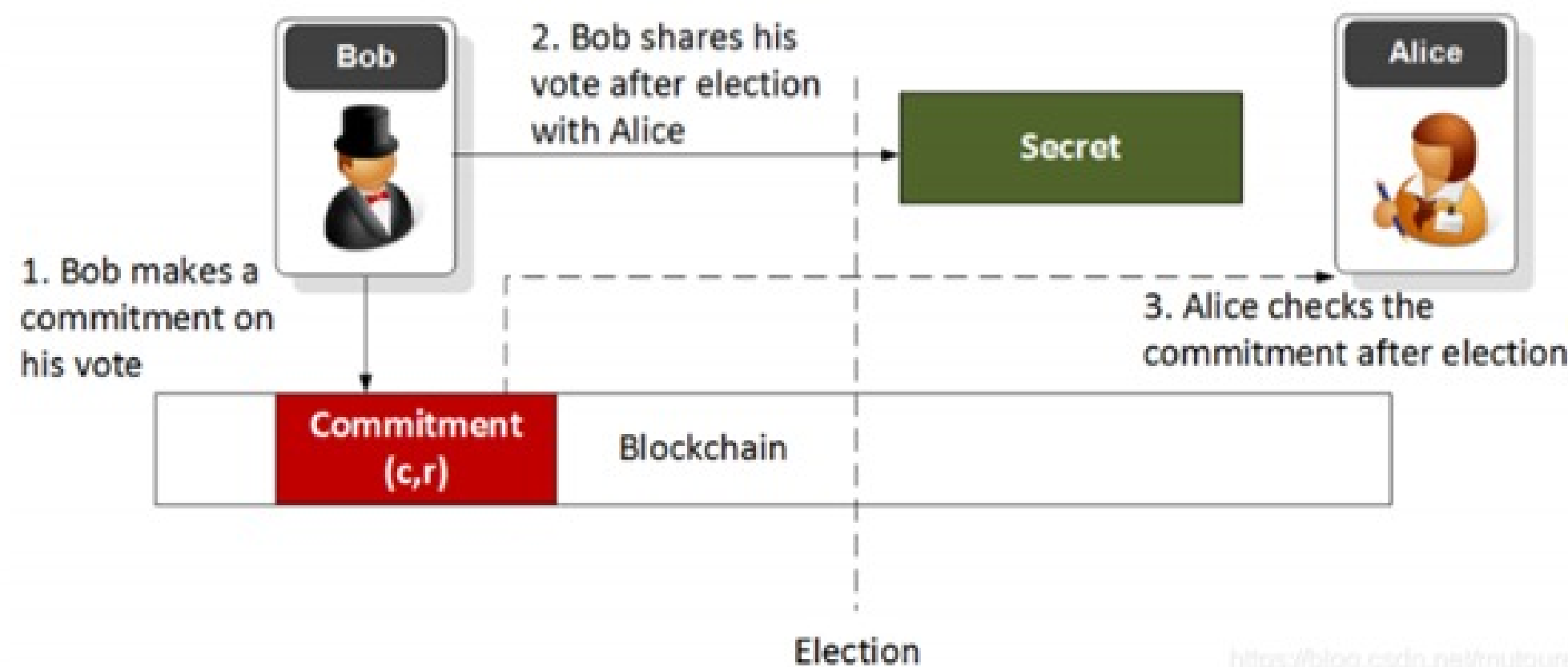
(1) 信息隐私性保护

Pedersen承诺允许我们承诺一个秘密信息，在未来的某个时候才真正透露出来。

(2) 电子投票

因为承诺具有绑定性，所承诺的秘密信息是不允许更改的，在未来透露出来的时候可以进行验证。因此，Pedersen承诺可以用于电子投票时候对所投的人进行承诺和事后校验。

Pedersen Commitment: Voting



3. 应用示例

(3) 隐私交易

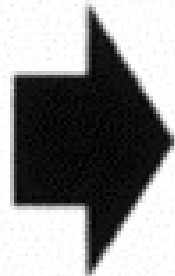
- Pedersen承诺还可以用来实现匿名保密交易等隐私交易协议。
- 假定在区块链上，Alice转给Bob一定数额的数字货币，在其他人不_{知道}数额和地址的情况下，使用Pedersen承诺可以保证这笔交易是有效的，使得任何人在区块浏览器上都查不到数额和地址信息。
- **如何在不知道交易金额的情况下验证这笔交易收支平衡呢？这就需要**我们利用Pedersen承诺来设计一个满足要求的协议：**第一，要能对拥有的金额、转账额度做出承诺；第二，基于做出的承诺可以进行交易收支平衡的验证。**

3. 应用示例

假定Alice有10元钱，要转给Bob的额度为7元，基于Pedersen承诺来实现的具体步骤如图所示。

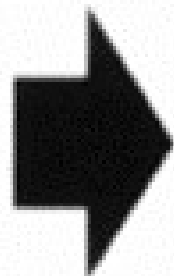
Alice

初始化，选择 G 、 H
生成随机数 r_{a1} 、 r_{a2}
计算承诺 C_{a1} 、 C_{a2} 、 r_a
 $C_{a1} = r_{a1} \times G + 10 \times H$
 $C_{a2} = r_{a2} \times G + 3 \times H$
 $r_a = r_{a1} - r_{a2}$
公布承诺和公钥
 C_{a1} 、 C_{a2} 、 r_a 、 G 、 H



Bob

生成随机数 r_b
计算
 $C_b = r_b \times G + 7 \times H$
 $K = (r_a - r_b) \times G$
验证Alice的承诺
 $C_{a1} - C_{a2} = r_a \times G + 7 \times H$
公开 C_b 、 K



其他验证者

验证
 $C_{a1} - C_{a2} = C_b + K$

Alice对自己的金额和余额做出承诺

Bob对收到的转账额度做出承诺

其他人可公开验证

3. 应用示例

➤ **三个承诺：** Alice的金额10元和3元的承诺 C_{a1} 和 C_{a2} ， Bob收到的7元转账的承诺 C_b 。

➤ **两个验证：**

• Bob（知道转账金额）对Alice的承诺的验证（转出7元），可通过下述式子验证

$$C_{a1}-C_{a2} = r_a G + 7 H, \quad r_a = (r_{a1}-r_{a2})$$

• 其他验证者可以通过Alice和Bob公开的承诺和相关信息进行整个交易的验证：

$$C_{a1}-C_{a2} = C_b + K, \quad K = (r_a-r_b) G$$

➤ **安全性：**显然，验证过程不会泄露双方的余额、交易的金额。攻击者可以通过计算 $K = C_{a1}-C_{a2}-C_b$ 得到 $7*H$ ，但是，因为椭圆曲线离散对数求解困难问题，其无法反推出7。同理，其他验证者也无法推断出Alice原有金额以及找零金额。

3. 应用示例

- 虽然Pederson承诺证明了数字之间的关系，但是并没有限制任何数字的取值区间，
如果提供的承诺出现了负数，则违背了余额不能为负的现实约束，使得整个数字货币体系不可用。因此，还需要对隐藏的数值进行范围证明。
- 不允许泄露隐私情况下证明数值的范围，就得依赖零知识证明了。

目 录

「01」

知识点一：哈希承诺

「02」

知识点二：加法同态承诺

「03」

知识点三：零知识证明

「04」

知识点四：零知识证明应用示例

1. 定义

零知识证明是在20世纪80年代初提出的，是一种涉及两方或更多方的协议，允许证明者能够在不向验证者提供任何有用的信息的情况下，使验证者相信某个论断是正确的。

简单例子：有一个缺口环形的长廊，出口和入口距离非常近（在目距之内），但走廊中间处有一道只能用钥匙打开的门，A要向B证明自己拥有该门的钥匙。采用零知识证明，B看着A从入口进入走廊，然后又从出口走出走廊，这时B没有得到关于这个钥匙的信息，但是完全可以证明A有钥匙。

1. 定义

零知识 (Zero-Knowledge, ZK) 证明允许证明方让验证方相信证明方自己知道一个满足 $C(x)=1$ 的 x , 但不会进一步泄漏关于 x 的任何信息。

这里的 C 是一个公开的谓词函数。谓词函数是一个判断式, 一个返回 `bool` 值的函数。该定义意味着一个零知识证明通常需要将证明过程转化为验证一个谓词函数是否成立的形式。

2. 性质

在零知识证明中，需要满足三个性质：

正确性：没有人能够假冒证明方P使这个证明成功。如果不满足这条性质，也就是证明方P不知道“知识”，再怎么证明，验证方V也很难相信证明方P拥有正确的知识。

完备性：如果证明方P和验证方V都是诚实的，并证明过程的每一步都进行正确的计算，那么这个证明一定是成功的。也就是说如果证明方P知道“知识”，那么验证方V会有极大的概率相信证明方P。

零知识性：证明执行完之后，验证方V只获得了“证明方P拥有这个知识”这条信息，而没有获得关于这个知识本身的任何信息。

3. 应用

零知识证明的应用场景很多，简要举例如下。

身份认证：可用于对用户进行身份验证，而无需交换密码等机密信息。比如，用户可以向网站证明，他拥有私钥，网站并不需要知道私钥的内容，可以通过验证这个零知识证明，确认用户的身份。

3. 应用

数据的隐私保护：在隐私场景中，根据零知识性，不泄漏交易的接收方、发送方、交易细节的前提下，满足特定业务的需求。比如：

- **买保险的时候，保险公司需要了解是否患有某种疾病，但是我不想让保险公司知道我的全部病历信息，那我可以证明给保险公司看，我没有相关疾病就足够了；**
- **在金融领域，抵押贷款申请人可以证明他们的收入在可接受的范围内，而不透露他们的确切工资；**
- 在区块链上，比特币和以太坊等区块链能保证链上数据的透明性使人人都是可以验证链上交易，这意味着参与者几乎没有了隐私，可能导致数据的非对称性，而零知识证明可以帮助保护区块链参与者的隐私权。

4. 交互式Schnorr协议

Schnorr（施诺尔）机制是一种基于离散对数难题的零知识证明机制。

证明者声称知道一个密钥 sk 的值，通过使用Schnorr加密技术，可以在不揭露 sk 的情况下，向验证者证明对 sk 的知情权。

依据椭圆曲线的离线对数难题，我们知道已知椭圆曲线 E 和生成元 G ，随机选择一个整数 a ，容易计算 $Q=a*G$ ，但是给定的 Q 和 G 计算 a 就非常困难。

4. 交互式Schnorr协议

Schnorr协议:

假设Alice 拥有一个秘密数字 a ，我们可以把这个数字当成私钥 sk ，然后把它映射到椭圆曲线群上的一个点 $a * G$ ，简写为 aG 。这个点我们把它当做公钥 PK 。

接下来，Alice向Bob证明她拥有 PK 对应的私钥 sk ，那么如何证明呢？

4. 交互式Schnorr协议

(1) 协议流程

给定 $pk=aG$ ，私钥 $sk=a$

承诺阶段： Alice产生一个随机数 r ，计算 $R=r*G$ 并发送 R 给Bob。

挑战阶段： Bob提供一个随机数 c 进行挑战，并将 c 发送给Alice。

回应挑战： Alice根据挑战数 c 计算 $z = r + a * c$ ，然后把 z 发给Bob。

验证阶段： Bob通过式子进行检验： $z*G \stackrel{?}{=} R + c*PK$ 。

由于 $z = r + c*a$ ，所以 $z*G = rG + c*(aG) = R + c*PK$ 。如果相等则证明Alice确实拥有私钥 sk ，但是验证者Bob并不能得到私钥 sk 的值，因此这个过程是零知识的。

4. 交互式Schnorr协议

(2) 安全性

由于椭圆曲线上的离散对数问题，知道 R 和 G 的情况下通过 $R=r*G$ 解出 r 是不可能的，所以保证了 r 的私密性。但是，整个过程是在证明者和验证者在私有安全通道中执行的。这是由于协议存在交互过程，只对参与交互的验证者有效。

5. 非交互式零知识证明

1. 设计要点

交互式零知识证明需要证明者和验证者时刻保持在线状态，非交互式零知识证明(Non-Interactive Zero Knowledge, NIZK)中，证明者仅需发送一轮消息即可完成证明。

但是，将交互式零知识证明直接转为非交互零知识证明会面临一些挑战，即减少随机挑战和回应挑战两个步骤后，如何防止证明方伪造证明成为设计的关键。

5. 非交互式零知识证明

为什么需要验证者回复一个随机数 c 呢？这是为了防止Alice造假。如果Bob不回复一个 c ，就变成一次性交互。因为 a 和 r 都是Alice自己生成的，她知道Bob会用 PK 和 R 相加然后再与 $z * G$ 进行比较，即 $z * G \stackrel{?}{=} \text{PK} + \text{R}$ 。所以她完全可以在不知道 a 的情况下构造： $\text{R} = r * G - \text{PK}$ 和 $z = r$ 。这样Bob的验证过程就变成： $z * G \stackrel{?}{=} \text{PK} + \text{R} \implies r * G \stackrel{?}{=} \text{PK} + r * G - \text{PK}$ 。这是永远成立的，所以这种方案并不正确。

目前主流的非交互式零知识证明的构造方法有两种：

- 一是基于ROM并利用Fiat-Shamir启发式实现的；
- 二是基于公共参考串CRS (Common Reference String)模型实现的。

5. 非交互式零知识证明

2. 非交互式Schnorr协议

非交互式Schnorr协议就是基于随机预言机并利用Fiat-Shamir变换实现的非交互式零知识证明协议，思路就是用公开的哈希函数的输出代替随机的挑战。

(1) 协议流程

承诺阶段： Alice均匀随机选择 r ，并依次计算

$$R=r*G, c=Hash(R,PK), z=r+c*sk,$$

然后生成证明 (R,z) 。

验证阶段： Bob(或者任意一个验证者)计算 $c=Hash(PK,R)$ ，验证

$$z*G \stackrel{?}{=} R+c*PK。$$

5. 非交互式零知识证明

(2) 安全性

为了不让Alice进行造假，在交互式Schnorr协议中需要Bob发送一个 c 值，并将 c 值构造进公式中。所以，在非交互式Schnorr协议中，如果Alice**选择一个无法造假并且大家公认的 c 值并将其构造进公式中**问题就解决了。**生成这个公认无法造假的 c 的方法是使用随机预言机。**一个密码学安全哈希函数是单向的，因此，虽然 c 是Alice计算的，但是Alice并没有能力实现通过挑选 c 来作弊。因为只要Alice**一旦产生 R ， c 就相当于固定下来了**。这样，就把三步Schnorr协议合并为一步。Alice可直接发送 (R, z) ，因为Bob拥有Alice的公钥 PK ，于是Bob可自行计算出 c 。然后验证 $z * G = c * PK + R$ 。

5. 非交互式零知识证明

3. 简洁零知识证明zkSNARK

- zkSNARK(zero-knowledge Succinct Non-interactive Arguments of Knowledge)就是基于公共参考字符串CRS模型实现的典型的非交互式零知识证明技术。zkSNARK中比较典型的协议有Groth10、GGPR13、Pinocchio、GRoth6、GKMMM18等。
- CRS模型是在证明者构造证明之前由一个受信任的第三方产生的随机字符串，CRS必须由一个受信任的第三方来完成，同时共享给证明者和验证者。CRS其实就把挑战过程中所要生成的随机数和挑战数，都预先生成好，然后基于这些随机数和挑战数生成他们对应的在证明和验证过程中所需用到的各种同态隐藏。之后，就把这些随机数和挑战数销毁。这些随机数和挑战数被称为toxic waste（有毒废物），因为如果他们没有被销毁的话，就可以伪造证明了。

5. 非交互式零知识证明

- zkSNARK(zero-knowledge Succinct Non-interactive Arguments of Knowledge)就 是一类基于CRS模型实现的典型的非交互式零知识证明技术，技术特征如下：
- **简洁性**：最终生成的证明具有简洁性，也就是说最终生成的证明足够小，并且与计算量大小无关。
- **无交互**：没有或者只有很少的交互。对于zkSNARK来说，证明者向验证者发送一条信息之后几乎没有交互。此外，zkSNARK还常常拥有“公共验证者”的属性，意思是在没有再次交互的情况下任何人都可以验证。
- **可靠性**：证明者在不知道**见证 (Witness, 私密的数据, 只有证明者知道)**的情况下，构造出证明是不可能的。
- **零知识**：验证者无法获取证明者的任何隐私信息。

1. 简洁的零知识证明zkSNARK

应用zkSNARK技术实现一个非交互式零知识证明应用的开发步骤大体如下：

- **定义电路：**将所要声明的内容的计算算法用算术电路来表示，简单地说，算术电路以变量或数字作为输入，并且允许使用加法、乘法两种运算来操作表达式。所有的NP问题都可以有效地转换为算术电路。
- **将电路表达为R1CS：**在电路的基础上构造约束，也就是R1CS（Rank- Constraint System，一阶约束系统），有了约束就可以把NP问题抽象成QAP（Quadratic Arithmetic Problem）问题，进而可以构建zkSNARKs。
- **完成应用开发：**
 - **生成密钥：**生成证明密钥(Proving Key)和验证密钥(Verification Key)，即CRS；
 - **生成证明：**证明方使用证明密钥和其可行解构造证明；
 - **验证证明：**验证方使用验证密钥验证证明方发过来的证明。

有关零知识证明，说法错误的是

- A 零知识性是指验证者能获得“证明方拥有这个知识”的信息，但没有获得知识本身的任何信息。
- B 交互式Schnorr协议可以通过应用随机预言机改造为非交互式零知识证明协议。
- C 交互式零知识证明需要证明者和验证者时刻保持在线状态，因此，在现实应用中基本没有合适的应用场景。
- D 交互式零知识证明直接删除随机挑战环节来改造为非交互式零知识证明容易引发私钥泄露问题。

目 录

「01」

知识点一：哈希承诺

「02」

知识点二：加法同态承诺

「03」

知识点三：零知识证明

「04」

知识点四：零知识证明应用示例

2. Libsnark框架

libsark是用于开发zkSNARK应用的C++代码库，由SCIPR Lab开发并采用商业友好的MIT许可证（但附有例外条款）在GitHub上（<https://github.com/scipr-lab/libsark>）开源。libsark框架提供了多个通用证明系统的实现，其中使用较多的是BCTV14a和Groth16。使用libsark框架需要有简单的C++语言基础，libsark源码目录如下：

— depends/	相关外部依赖
— tinyram_examples/	包含TinyRAM（一种比R1CS更高阶的描述问题的语言）的例子
— libsark/	主题代码目录
— common/	定义和实现了一些通用的数据结构，例如默克尔树、稀疏向量等
— gadgetlib1/	抽象出一层以便构建R1CS（基于模板），提供较丰富的gadget
— gadgetlib2/	接口不基于模板，文档更好、更易用，但gadget较少
— knowledge_commitment/	在 multiexp的基础上，引入pair概念
— reductions	各种不同描述语言之间的转化
— relations	各种NPC问题描述语言的接口（包括R1CS、USCS等）
— zk_proof_systems	各种证明系统（包括Groth16、GM17等）

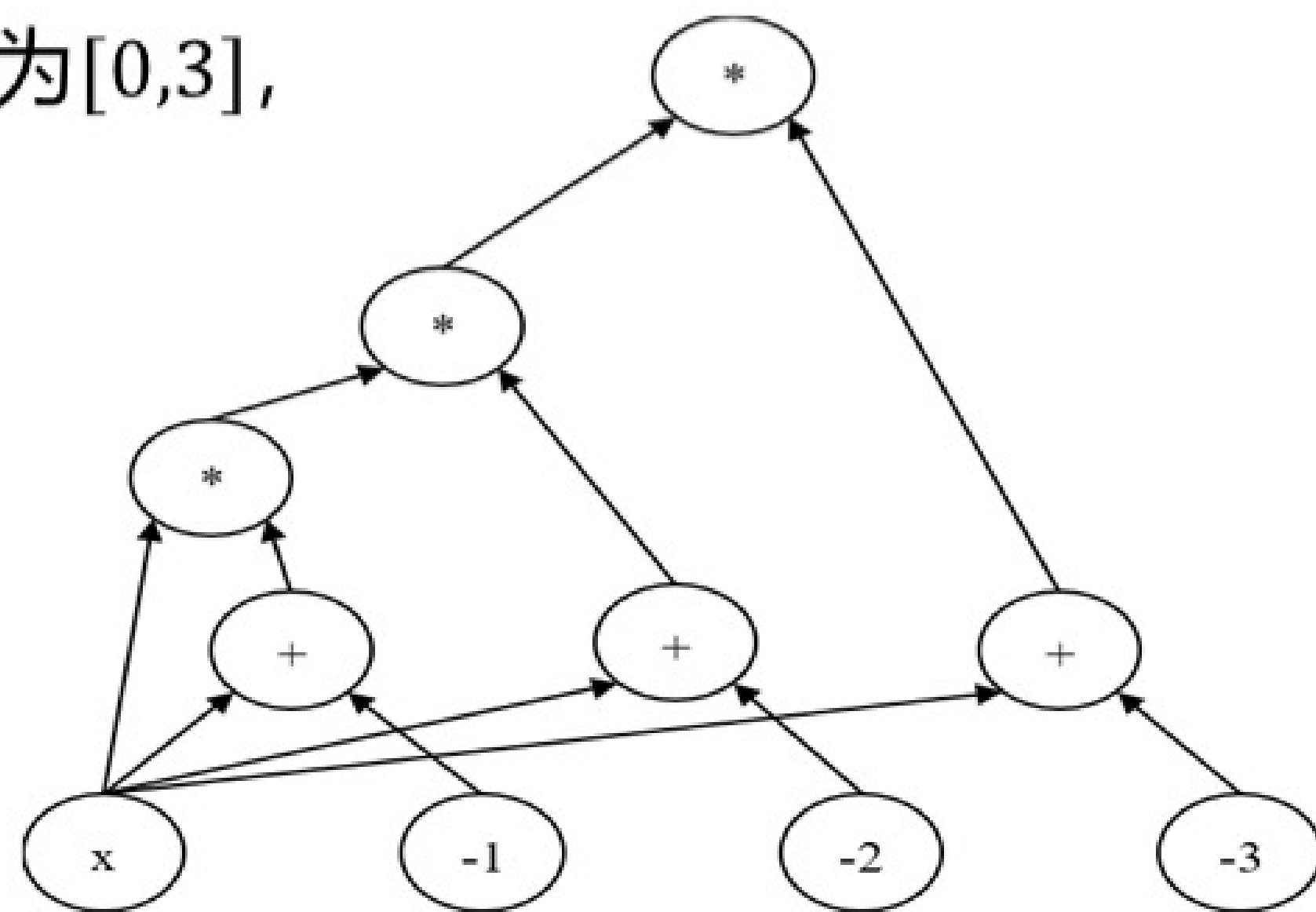
3. 应用示例

实验3.1 假设证明方有一个整数 x ，他希望向验证方证明这个整数 x 的取值范围为 $[0,3]$ 。

(1) 将待证明的命题表达为R1CS

● 用算术电路表示待证明问题

在计算复杂性理论中，计算多项式的最自然计算模型就是算术电路。如果想要约束整数 x 必须取值于 $[a, b]$ 的话，可以用如下公式来约束它： $(x - a) * (x - (a + 1)) * \dots * (x - b) = 0$ 。对于待证明的问题证明整数 x 的取值范围为 $[0,3]$ ，可以写出如下约束： $x(x - 1)(x - 2)(x - 3) = 0$ ，该约束对应的算术电路如图所示。



3. 应用示例

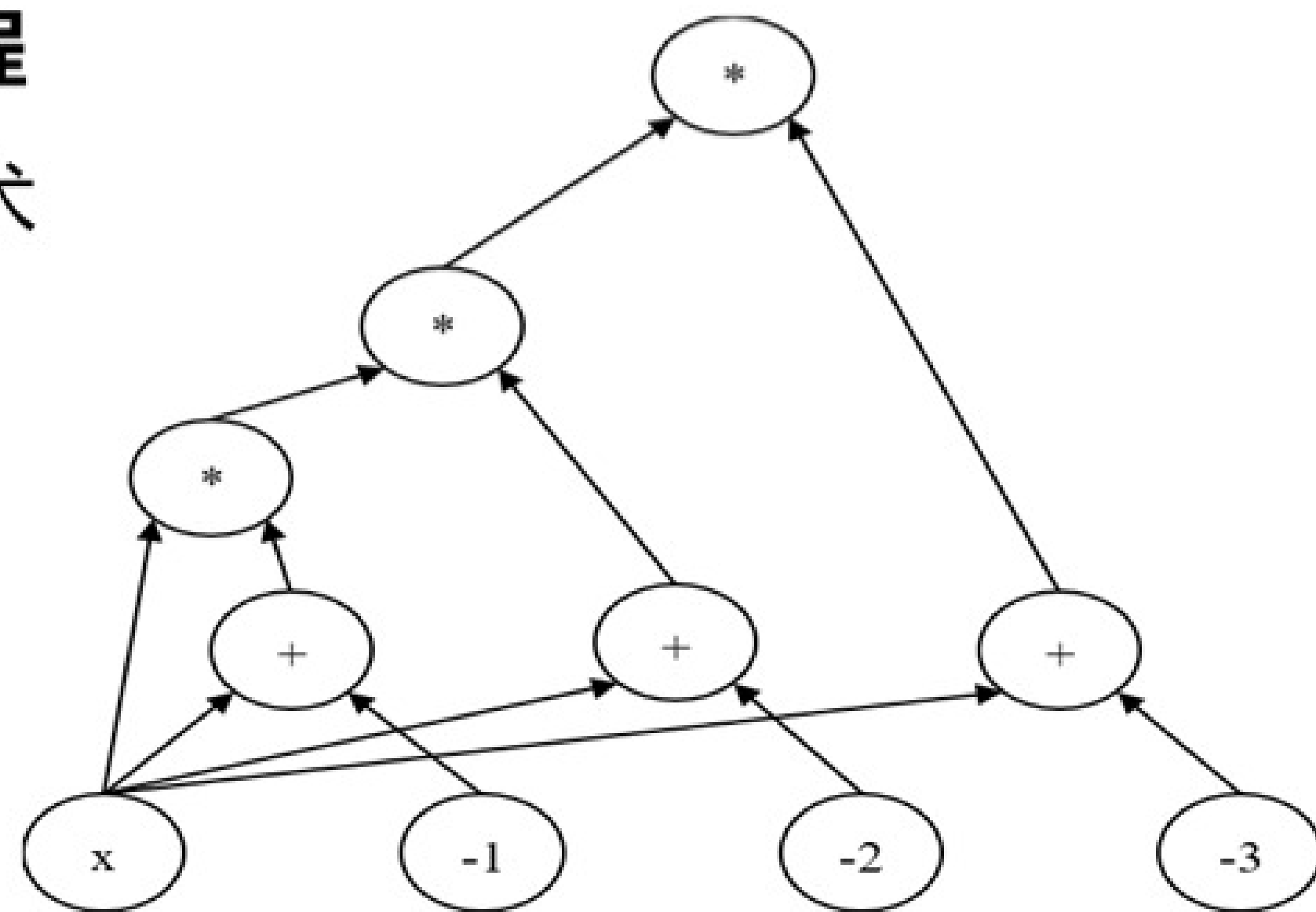
实验3.1 假设证明方有一个整数 x ，他希望向验证方证明这个整数 x 的取值范围为 $[0,3]$ 。

(1) 将待证明的命题表达为R1CS

● 用算术电路表示待证明问题

可以用 $C(v, w)$ 来表示上述电路，其中 v 为公有输入，表达了想要证明的问题的特性和一些固定的环境变量，所有人都知道 v ； w 为私密输入，只有证明方才会知道。

将待证明命题转换为算术电路之后，**就可以将证明过程转换为证明 $C(v, w) = 0$** ，即在证明方和验证方已知算术电路 C 的输出为0，并且公有输入为 v 的情况下，证明方需要证明他拥有能构成电路输出为0的私密输入值 w 。



3. 应用示例

2) 用R1CS描述电路

首先，将算数电路拍平成多个 $x = y$ 或者 $x = y (op) z$ 形式的等式，其中 op 可以是加、减、乘、除运算符中的一种。对于 $x(x - 1)(x - 2)(x - 3) = 0$ ，可以拍平成如下几个等式：

$$w_1 = x - 1$$

$$w_2 = x - 2$$

$$w_3 = x - 3$$

$$w_4 = x * w_1$$

$$w_5 = w_2 * w_4$$

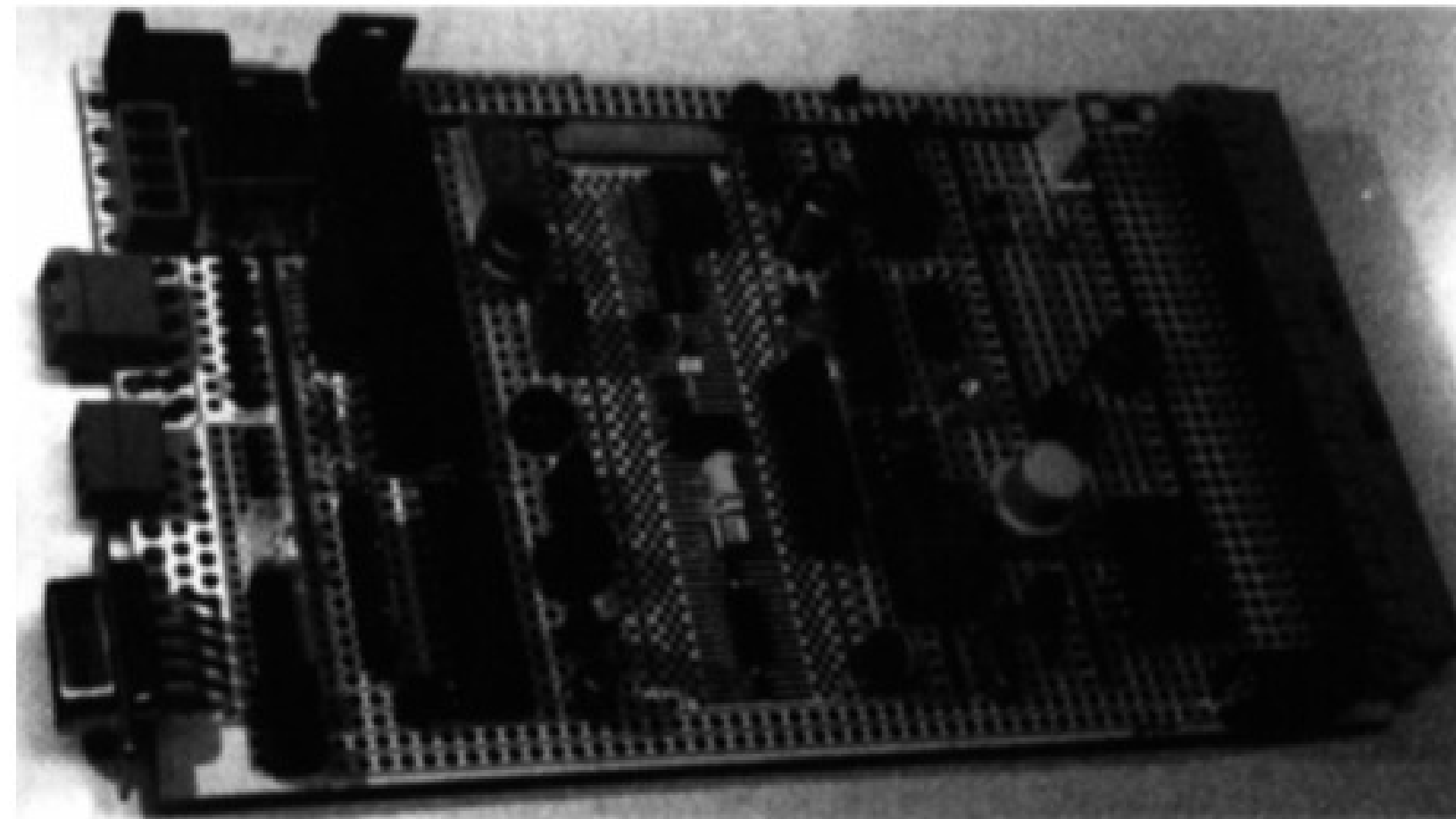
$$out = w_3 * w_5$$

3. 应用示例

3) 使用原型板搭建电路

然后，使用原型板protoboard搭建电路。

在电气工程中，原型板（Protoboard）是用于连接电路和芯片的，如图所示。



将待证明的命题用电路表示，并用R1CS描述电路之后，就可以**构建一个protoboard**。
protoboard，也就是原型板或者面包板，可以用来快速搭建算术电路，把所有变量、组件和约束关联起来。

3. 应用示例

因为在初始设置、证明、验证三个阶段都需要构造面包板，所以这里将下面的代码放在一个公用的文件common.hpp中供三个阶段使用。

```
//代码开头引用了三个头文件：第一个头文件是为了引入default_r1cs_gg_ppzksnark_pp 类型；第二个则为了引入证明相关的各个接口；pb_variable 则是用来定义电路相关的变量。
#include <libsnark/common/default_types/r1cs_gg_ppzksnark_pp.hpp>
#include <libsnark/zk_proof_systems/ppzksnark/r1cs_gg_ppzksnark/r1cs_gg_ppzksnark.hpp>
#include <libsnark/gadgetlib1/pb_variable.hpp>
using namespace libsnark;
using namespace std;
typedef libff::Fr<default_r1cs_gg_ppzksnark_pp> FieldT; //定义使用的有限域
//定义创建面包板的函数
protoboard<FieldT> build_protoboard(int* secret){
    default_r1cs_gg_ppzksnark_pp::init_public_params(); //初始化曲线参数
    protoboard<FieldT> pb; //创建面包板
    //定义所有需要外部输入的变量以及中间变量
    pb_variable<FieldT> x;
    pb_variable<FieldT> w_1, w_2, w_3, w_4, w_5, out;
```

//下面将各个变量与protoboard连接，相当于把各个元器件插到“面包板”上。allocate()
()函数的第二个string类型变量仅是用来方便DEBUG时的注释，方便DEBUG时查看日志。

```
out.allocate(pb, "out");
```

```
x.allocate(pb, "x");
```

```
w_1.allocate(pb, "w_1");
```

```
w_2.allocate(pb, "w_2");
```

```
w_3.allocate(pb, "w_3");
```

```
w_4.allocate(pb, "w_4");
```

```
w_5.allocate(pb, "w_5");
```

//定义公有的变量的数量，set_input_sizes(n)用来声明与protoboard连接的public变量的个数n。在这里n=1，表明与pb连接的前n = 1个变量是public的，其余都是private的。因此，要将public的变量先与pb连接（前面out是公开的）。

```
pb.set_input_sizes(1);
```

```
//为公有变量赋值
```

```
pb.val(out)=0;
```

//确定变量间的约束关系。调用protoboard 的add_r1cs_constraint()函数，为pb添加形如 $a * b = c$ 的约束，即r1cs_constraint<FieldT>(a, b, c)中参数应该满足 $a * b = c$ 。

```
pb.add_r1cs_constraint(r1cs_constraint<FieldT>(x-1, 1, w_1)); // x-1= w_1
pb.add_r1cs_constraint(r1cs_constraint<FieldT>(x-2, 1, w_2)); // x-2= w_2
pb.add_r1cs_constraint(r1cs_constraint<FieldT>(x-3, 1, w_3)); // x-3= w_3
pb.add_r1cs_constraint(r1cs_constraint<FieldT>(x, w_1, w_4)); // x*w_1=w_4
pb.add_r1cs_constraint(r1cs_constraint<FieldT>(w_2, w_4, w_5)); //w_2*w_4=w_5
pb.add_r1cs_constraint(r1cs_constraint<FieldT>(w_3, w_5, out)); //w_3*w_5=out
//证明者在生成证明阶段传入私密输入，为私密变量赋值，其他阶段为NULL
if (secret!=NULL) {
    pb.val(x)=secret[0];
    pb.val(w_1)=secret[1];
    pb.val(w_2)=secret[2];
    pb.val(w_3)=secret[3];
    pb.val(w_4)=secret[4];
    pb.val(w_5)=secret[5];
}
return pb; }
```

3. 应用示例

实验3.1 假设证明方有一个整数 x ，他希望向验证方证明这个整数 x 的取值范围为 $[0,3]$ 。

(2) 生成证明密钥和验证密钥

在这个阶段，我们使用生成算法为该命题生成公共参数（证明密钥和验证密钥），并把生成的证明密钥和验证密钥输出到对应文件中保存。其中，证明密钥供证明者使用，验证密钥供验证者使用。编写代码如下，将这段代码放在mysetup.cpp中。

```
protoboard<FieldT> pb=build_protoboard(NULL); //构造面包板
const r1cs_constraint_system<FieldT> constraint_system = pb.get_constraint_system();
    //生成证明密钥和验证密钥
const r1cs_gg_ppzksnark_keypair<default_r1cs_gg_ppzksnark_pp> keypair = r1cs_gg_ppzk
snark_generator<default_r1cs_gg_ppzksnark_pp>(constraint_system);
    //保存证明密钥到文件pk.raw
fstream pk("pk.raw",ios_base::out);
pk<<keypair.pk;
pk.close();
```

3. 应用示例

(3) 证明方使用证明密钥和其可行解构造证明

在定义面包板时，我们已为public input提供具体数值，在**构造证明阶段，证明者只需为private input提供具体数值，进而通过prover函数生成证明**。生成的证明保存到proof.raw文件中供验证者使用。编写代码如下，将这段代码放在myprove.cpp中。

```
int main()
{
//输入秘密值x
int x;
cin>>x;
//确定私密输入
int secret[6];
secret[0]=x;
secret[1]=x-1;
secret[2]=x-2;
secret[3]=x-3;
secret[4]=x*(x-1);
secret[5]=x*(x-1)*(x-2);
```

```
protoboard<FieldT> pb=build_protoboard(secret); //构造面包板
const r1cs_constraint_system<FieldT> constraint_system = pb.get_constraint_system();
fstream f_pk("pk.raw",ios_base::in); //加载证明密钥
r1cs_gg_ppzksnark_proving_key<libff::default_ec_pp>pk;
f_pk>>pk;          f_pk.close();

const r1cs_gg_ppzksnark_proof<default_r1cs_gg_ppzksnark_pp> proof = r1cs_gg_ppzk
snark_prover<default_r1cs_gg_ppzksnark_pp>(pk, pb.primary_input(), pb.auxiliary_in
put()); //生成证明
//将生成的证明保存到proof.raw文件
fstream pr("proof.raw",ios_base::out);      pr<<proof;      pr.close();
return 0; }
```

3. 应用示例

(4) 验证方使用验证密钥验证证明方发过来的证明

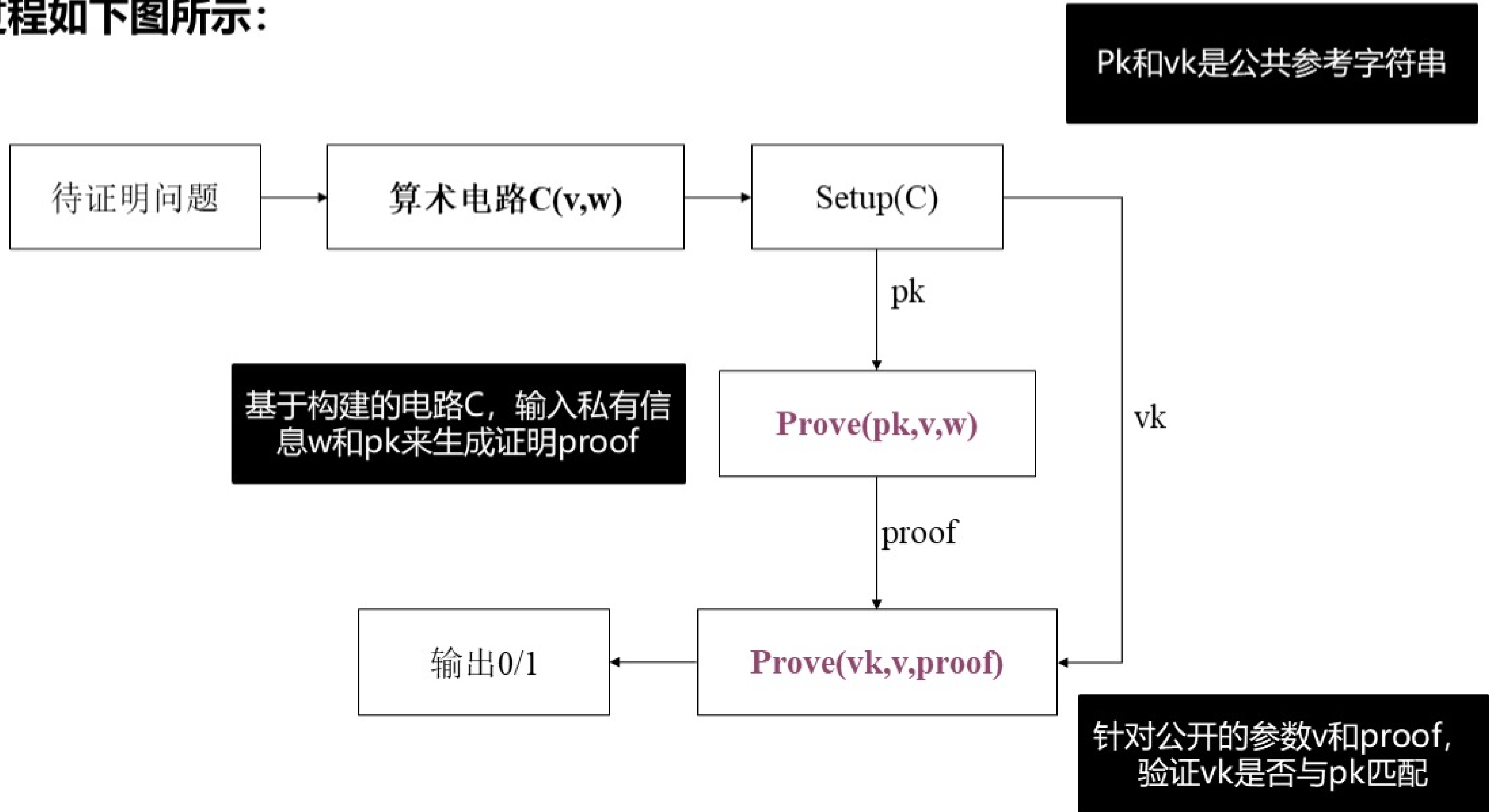
最后我们使用verifier函数校验证明。如果verified = 1则说明证明验证成功。编写代码如下，将这段代码放在myverify.cpp中。

```
int main()
{
//构造面包板
protoboard<FieldT> pb=build_protoboard(NULL);
const r1cs_constraint_system<FieldT> constraint_system = pb.get_constraint_system();
//加载验证密钥
fstream f_vk("vk.raw",ios_base::in);
r1cs_gg_ppzksnark_verification_key<libff::default_ec_pp>vk;
f_vk>>vk;
f_vk.close();
```

```
//加载银行生成的证明
fstream f_proof("proof.raw",ios_base::in);
r1cs_gg_ppzksnark_proof<libff::default_ec_pp>proof;
f_proof>>proof;
f_proof.close();
//进行验证
bool verified = r1cs_gg_ppzksnark_verifier_strong_IC<default_r1cs_gg_ppzksnark_pp>(vk, pb.primary_input(), proof);
cout<<"验证结果:"<<verified<<endl;
return 0;
}
```

3. 应用示例

整体过程如下图所示：



4.可复用的电路Gadget

在libsnark项目中使用R1CS仍然是比较复杂的一件事情。

如果能将一些常用的算术电路预制到库中，这将给编程带来很大的方便。

gadgetlib1和gadgetlib2就是实现该功能的工具包，其中包含了一些基本运算的R1CS，
比如 **sha256在内的哈希计算、默克尔树、pairing 等电路实现**。

在原型板中使用Gadget非常简单，这里不赘述，自己通过教材了解。

谢 谢

